



EMBEDDED SOFTWARE ENGINEERING

Session 2 – Git, CubelIDE & the Superloop

Laurent Fiack – laurent.fiack@ensea.fr

Bureau D212

Today

- Last time: a fast C refresher. Today: the tools, then the architecture.
- Git – just enough to pull/push the "bidule" project and commit your own work
- CubelIDE 2.x – importing a CMake project (you knew 1.x, with CubeMX built in)
- The **superloop**: the architecture every embedded project in this course starts from

Today

- 1 Git
- 2 CubeIDE 2.x
- 3 A First Naive Blink
- 4 "Just Put Everything in the Interrupt"?
- 5 Foreground / Background

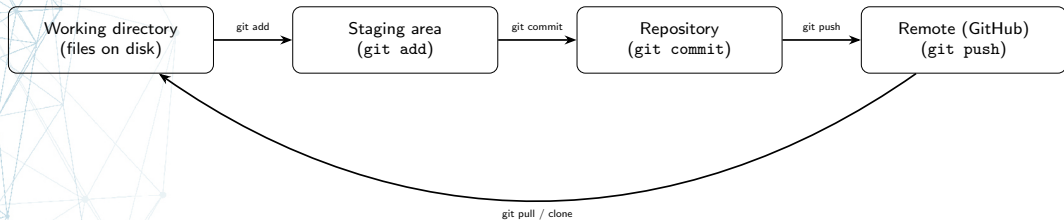
Why Git?

- Every project you'll touch this year – and every real one after – lives in a repository
- Answers questions you will actually ask yourself: *"what did I change?"*, *"why did this used to work?"*, *"how do I get back to yesterday's working version?"*
- Lets you and your teammates work on the same project without overwriting each other

Today's goal

Just enough to clone/pull the "bidule" project, commit your own work, and not lose anything. Branches, collaboration, and conflicts come back in depth during the many project sessions ahead.

Three places your code can be



- You edit files – nothing is tracked until you add it
- `commit` takes a permanent snapshot of what's staged, with a message
- `push/pull` synchronize your local repository with GitHub

The everyday commands

<code>git clone <url></code>	<i># get a copy of a remote repository</i>
<code>git status</code>	<i># what changed? what's staged?</i>
<code>git add <file></code>	<i># stage a file (or `git add -A` for everything)</i>
<code>git commit -m "message"</code>	<i># snapshot what's staged</i>
<code>git log --oneline</code>	<i># history, one line per commit</i>
<code>git push</code>	<i># send local commits to the remote</i>
<code>git pull</code>	<i># fetch and merge the remote's new commits</i>

- `git status` is your best friend – run it constantly, it never lies

GitHub: the remote repository

- GitHub hosts a copy of the repository – it is *not* Git itself, just the most common place to host one
- Gives you: a backup off your laptop, a way to share code, a place to review changes (pull requests)
- For this course: clone the "bidule" project from GitHub, work locally, commit as you go, push when you want a backup or to share

Good habits

```
# .gitignore -- tell git what NOT to track  
build/  
*.o  
*.elf
```

- **.gitignore**: build artifacts don't belong in the repository – they're regenerated, and they turn every diff into noise
- **Commit messages**: say *why*, not just *what* – "fix off-by-one in scroll bar", not "update"
- **Frequency**: small, frequent commits beat one giant commit at the end of the session – each one is a checkpoint you can come back to

Branches, collaboration, conflicts

Coming back in depth once you're actually working with a partner on a shared project.

From CubelIDE 1.x to 2.x

- CubelIDE 1.x: CubeMX (pin/clock/peripheral configuration) is built in – the .ioc file drives an Eclipse-managed project directly
- CubelIDE 2.x: the IDE now expects a standard **CMake** project – CubeMX still generates the .ioc-driven sources, but the project itself is just CMake, importable by any CMake-aware tool
- Why it matters here: the same project (Makefile/CMakeLists.txt) builds from the command line, from CubelIDE, or from any other editor – no lock-in

Importing a CMake project into CubelIDE 2.x

- File → Import → Existing CMake Project (not "STM32 project" – that's the 1.x-style flow)
- Point it at the folder containing CMakeLists.txt (e.g. td2/)
- CubelIDE configures the toolchain from CMakePresets.json, then builds and lets you flash/debug exactly as before

Live demo

We do this together, right now, on the td2 project – everything after this slide is what we'll build inside it.

A blink, the obvious way

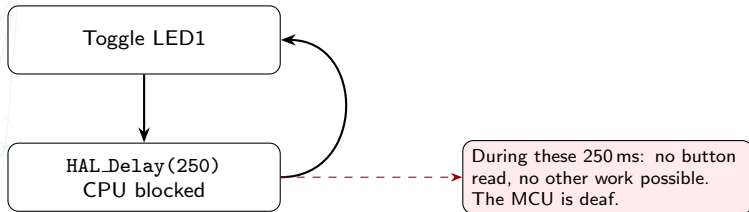
```
/* main.c -- infinite loop */  
while (1) {  
    HAL_GPIO_TogglePin(LED1_GPIO_Port, LED1_Pin);  
    HAL_Delay(250);  
}
```

- Compiles, flashes, LED1 blinks every 250 ms – looks done
- Now add: LED2 toggles on button SW1, LED3 toggles on button SW2

What's the problem?

- `HAL_Delay(250)` does not give the CPU back – it spins until 250 ms have elapsed
- For that entire time, the processor executes *nothing else*: no button read, no sensor poll, no UART, nothing
- Press SW1 during those 250 ms and... nothing happens. The press is simply never seen.
- This gets worse, not better, as you add more things the board should react to

Flowchart: the blocking blink



First fix: react to buttons in an interrupt

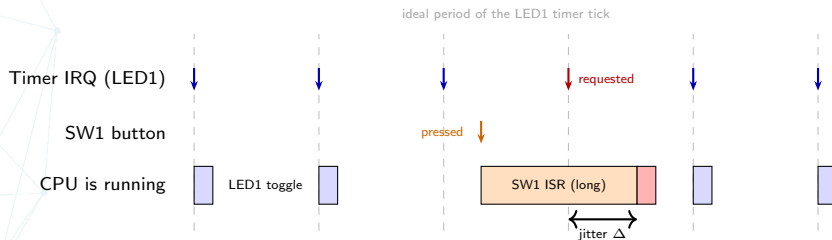
```
/* main.c -- CODE 0 */  
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {  
    switch (GPIO_Pin) {  
        case BTN_SW1_Pin: HAL_GPIO_TogglePin(LED2_GPIO_Port, LED2_Pin); break;  
        case BTN_SW2_Pin: HAL_GPIO_TogglePin(LED3_GPIO_Port, LED3_Pin); break;  
    }  
}
```

- An EXTI interrupt fires the instant the button is pressed – no more missed presses
- Tempting next thought: get rid of HAL_Delay too – drive LED1 from a periodic **timer interrupt**, and do *all* the logic inside interrupt handlers

Why not put everything in interrupts?

- An interrupt handler (ISR) runs to completion before anything of equal or lower priority can run – including *other* interrupts
- If a button ISR does real work (debounce, update a display over I²C, run some computation...), it can run long
- While it runs, the periodic timer interrupt that should toggle LED1 is stuck waiting – LED1's timing jitters
- You have not removed the blocking problem – you've just moved it from `main()` into the interrupts themselves

Chronogram: one long ISR delays everything else



- The tick at $t=3$ can't run until the SW1 handler finishes at $t=3.55$ – LED1 toggles **late**, by Δ

What this tells us

- Interrupts buy you *responsiveness to the event*, not free, unlimited processing time
- Rule of thumb: **keep interrupt handlers short**. Record that something happened; do the actual work elsewhere
- "Elsewhere" is exactly what we still have: the `main()` loop

The pattern

- **Foreground** (interrupts): reacts instantly to hardware events, but only does the bare minimum – typically, record that something happened
- **Background** (the `main()` loop – the *superloop*): runs continuously, checks what the foreground recorded, and does the actual work – the "business logic"
- Nothing here blocks: no `HAL_Delay`, no ISR doing real work – just fast checks, over and over

Foreground: minimal interrupt handler

```
static volatile bool sw1_pressed = false;
static volatile bool sw2_pressed = false;

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    switch (GPIO_Pin) {
        case BTN_SW1_Pin: sw1_pressed = true; break;
        case BTN_SW2_Pin: sw2_pressed = true; break;
    }
}
```

- That's it. The ISR's entire job is to raise a flag – volatile because it's written in an interrupt and read in main()

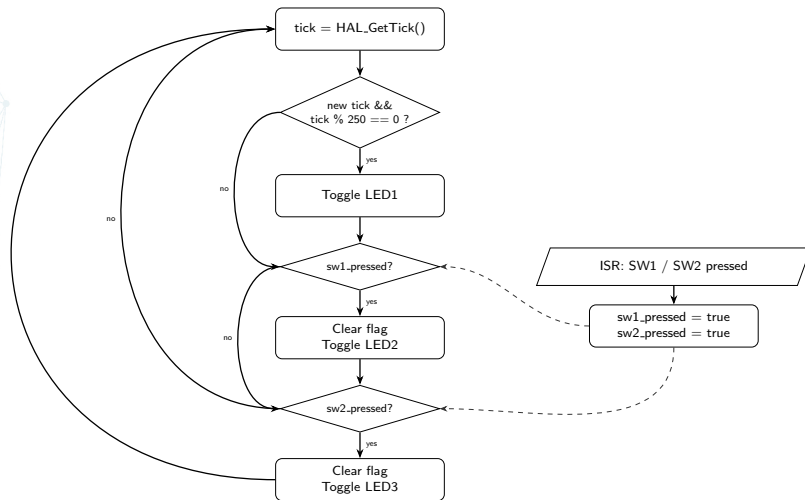
Background: the superloop does the work

```
uint32_t last_tick = 0;

while (1) {
    uint32_t tick = HAL_GetTick();
    if (last_tick != tick) {
        last_tick = tick;
        if (0 == (tick % LED_BLINK_PERIOD)) {
            HAL_GPIO_TogglePin(LED1_GPIO_Port, LED1_Pin);
        }
    }
    if (sw1_pressed) {
        sw1_pressed = false;
        HAL_GPIO_TogglePin(LED2_GPIO_Port, LED2_Pin);
    }
    if (sw2_pressed) {
        sw2_pressed = false;
        HAL_GPIO_TogglePin(LED3_GPIO_Port, LED3_Pin);
    }
}
```

- LED1 blinks by checking HAL_GetTick() – no blocking wait, ever
- The loop is free the rest of the time to check flags, and later, to do far more

Flowchart: the superloop solution



Foreground vs. background: takeaway

	Foreground (ISR)	Background (superloop)
Triggered by	hardware event	runs continuously
Job	record <i>that</i> it happened	decide <i>what to do about it</i>
Duration	as short as possible	no artificial blocking
Example here	sw1_pressed = true	toggle LED1/LED2/LED3

- This is exactly `td2's main.c` – what we just built together, live, in CubeIDE
- Next session: this same superloop gets its first proper structure – separating what talks to the hardware (BSP) from what decides (the application)

Exercise : From encoder to Neopixel

Goal

Change the neopixel color (hue) with the encoder.

Autonomous exercise – start from td2, work through these on your own:

- 1 How do you start the encoder's timer in encoder mode, and read its current count?
- 2 Use `hsv_to_rgb` (next slide), to convert hue to RGB. $S = 250$, $V = 50$.
- 3 Use neopixel lib from last TD.
- 4 Make sure DMA transfert has fully finished – and WS2812's reset time has passed – before the next call.
- 5 How do you avoid sending data when the encoder hasn't moved?

HSV to RGB

```
static void hsv_to_rgb(int h, int s, int v, uint8_t *r, uint8_t *g, uint8_t *b) {  
    h = ((h % 360) + 360) % 360;  
    if (s <= 0) {  
        *r = *g = *b = (uint8_t)v;  
        return;  
    }  
  
    int region = h / 60;  
    int rem = h % 60;  
    int p = (v * (255 - s)) / 255;  
    int q = (v * (255 - (s * rem) / 60)) / 255;  
    int t = (v * (255 - (s * (60 - rem)) / 60)) / 255;  
  
    switch (region) {  
        case 0: *r = (uint8_t)v; *g = (uint8_t)t; *b = (uint8_t)p; break;  
        case 1: *r = (uint8_t)q; *g = (uint8_t)v; *b = (uint8_t)p; break;  
        case 2: *r = (uint8_t)p; *g = (uint8_t)v; *b = (uint8_t)t; break;  
        case 3: *r = (uint8_t)p; *g = (uint8_t)q; *b = (uint8_t)v; break;  
        case 4: *r = (uint8_t)t; *g = (uint8_t)p; *b = (uint8_t)v; break;  
        default: *r = (uint8_t)v; *g = (uint8_t)p; *b = (uint8_t)q; break;  
    }  
}
```